

Task descriptions

void TaskLinduinoCVComms(void *pvParameters)

This task handles communication with the **Linduino (DC2026)** over an **isolated UART**.

Communication scheme

ESP32 → Linduino (request)

- The ESP32 requests **12× cell voltages** for a given segment and sends the **balancing configuration** for that segment.
- The balance configuration is stored as a uint16_t, where each bit represents whether balancing is ON/OFF for a module:
BAL[segment_index] = 0x0XXX (LSB corresponds to module 0).
- Each request packet is **1 byte**, encoded as:
 - **Segment ID (3 bits)**
Identifies the requested segment (1–6).
 - **Balance Index (2 bits)**
Selects which 3-bit slice of the 12-bit balance word:
 - 0 → BAL[2:0]
 - 1 → BAL[5:3]
 - 2 → BAL[8:6]
 - 3 → BAL[11:9]
 - **Balance Config (3 bits)**
Balancing ON/OFF for the selected 3 modules.

Notes

- Four request bytes are sent (one per balance index).
- Bytes are sent 1 ms apart.
- Packet is self-identifying; order does not matter.

Linduino → ESP32 (response)

- The Linduino receives the 4 bytes and reconstructs the full **12-bit balance configuration** for the requested segment.
- It responds with **26 bytes** in the following format:
 - Byte 0 : Segment ID (Same as request, extended to 8 bits (0x01 - 0x06))
 - Byte 1–24 : 12 × uint16_t Cell Voltages (big-endian)
 - Byte 25 : CRC (8-bit)

Details

- Cell voltages start from **module 0**.
- Each cell voltage is **uint16_t (actual CellV * 10000)**
- CRC is calculated over the **12 × 16-bit cell voltage data** only.
- CRC polynomial: 100000111

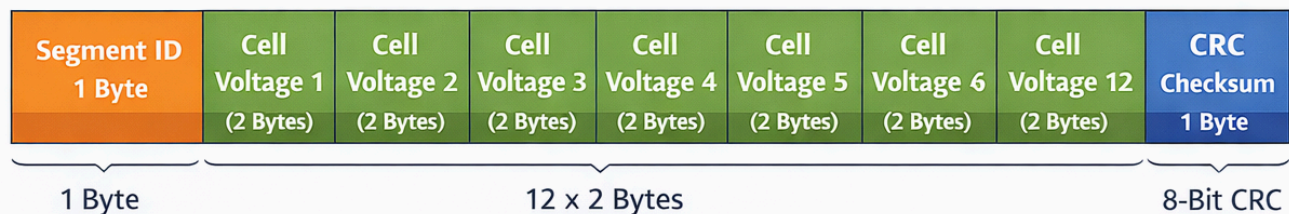
ESP32 receive handling

- The ESP32 reads the 26-byte response.
- It first checks that the **segment index matches** the requested segment.
 - If not, the serial stream is flushed and the packet is ignored.
- The CRC is verified.
- If valid, the received cell voltages are pushed to the **cell-voltage queue**.
- The ESP32 then requests data for the **next segment**, and the cycle repeats.

ESP32 to Linduino – Request Packet (4 × 1 Byte)



Linduino to ESP32 – Response Packet (26 Bytes)

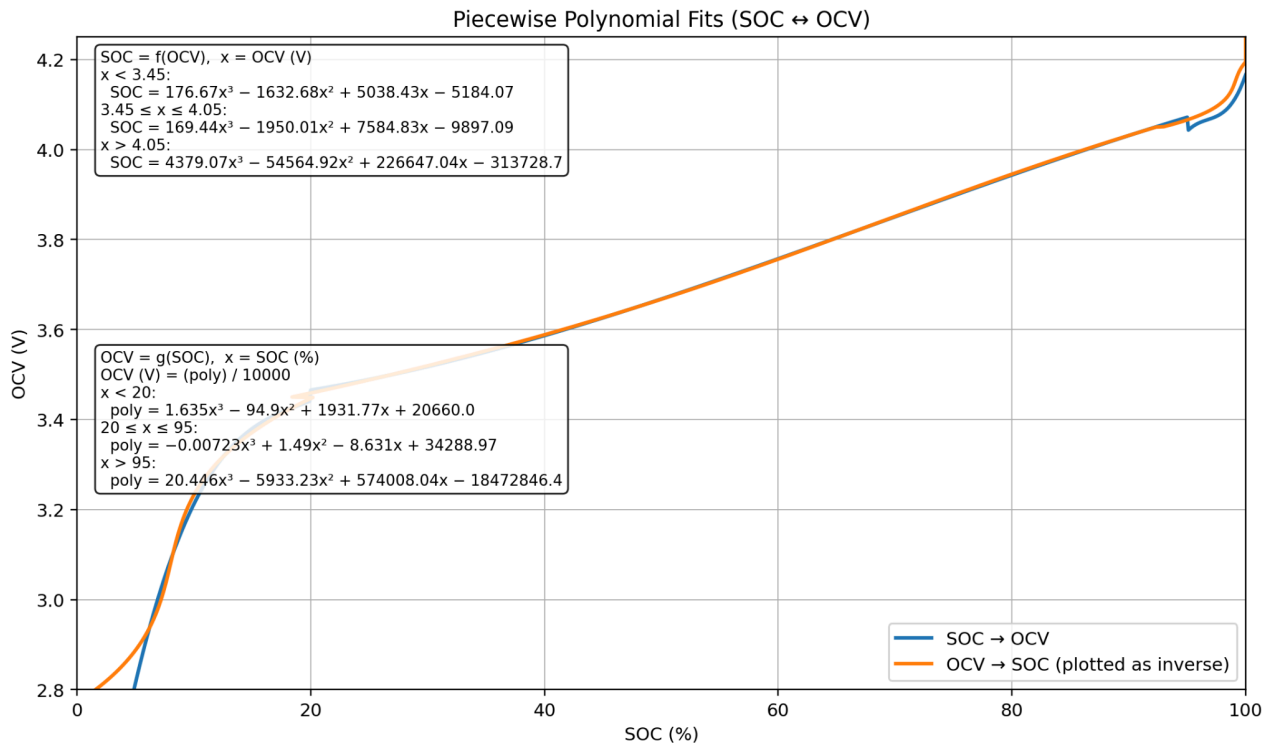


void TaskUpdateSOC(void *pvParameters)

This task updates SOC using two methods and then fuses them.

Method 1: SOC1 (voltage-based)

- The task estimates OCV using:
 - $OCV = CellV + (TS_current + I_balance_if_ON) * DCIR$
- It then estimates SOC from OCV using your piecewise 3rd-order fit:
 - $SOC1 = OCV2SOC(OCV)$



Method 2: SOC2 (coulomb counting)

- SOC2 is updated using coulomb counting:
 - $SOC2 = SOC2_prev - (TS_current * \Delta t) / MOD_Capacity$
- Here MOD_Capacity = 25Ah, and $\Delta t = (current_update_time - last_update_time)$.

Fusion: complementary filter

- SOC1 is trusted only at low current, so the blend factor is:
 - $\alpha = \max(0, (1 - \text{abs}(TS_current)/I_threshold))$
- Final SOC is:
 - $SOC_final = \alpha * SOC1 + (1 - \alpha) * SOC2$

Notes

- SOC2 needs initialization, so on the first update:
 - $SOC1 = SOC2 = OCV2SOC(OCV)$
- If the actual module capacity is less than MOD_Capacity (25Ah), SOC2 will change slower than SOC1 during discharge.
- The ratio $\Delta SOC2 / \Delta SOC1$ can be used as a rough SOH estimate (since SOC2 assumes ideal capacity).
- There are small discontinuities at the boundaries of the piecewise polynomial fit.
- Even with these discontinuities, the fits were chosen such that **within each region**, repeated inter-conversion **OCV** → **SOC** → **OCV** → ... will still **converge**, even in the presence of floating-point rounding error.
- This task starts only after the first round of cellVs for all 6 segments have been received (TaskLinduinoCVComms gives a notification)

void TaskUpdateDCIR(void *pvParameters)

This task updates DCIR dynamically during operation.

DCIR update equation

DCIR is estimated as:

- $DCIR_{new} = (SOC2OCV(SOC_{final}) - CellV) / (TS_{current} + I_{balance_if_ON})$

Limits + jump filtering

The new DCIR is accepted only if:

- $DCIR_MIN_LIMIT < DCIR_{new} < DCIR_MAX_LIMIT$
- AND $abs(DCIR_{new} - DCIR_{old}) < DCIR_JUMP_LIMIT$
- Example limits:
 - $DCIR_MIN_LIMIT = 2500 \mu\Omega$
 - $DCIR_MAX_LIMIT = 5000 \mu\Omega$
 - $DCIR_JUMP_LIMIT = 1000 \mu\Omega$

Smoothing

If the limits pass, DCIR is updated using **exponential smoothing**:

- $DCIR = beta * DCIR_{new} + (1 - beta) * DCIR_{old}$

Note

- The clipping + jump limit + smoothing is mainly to reduce noise and to average out effects like cell polarization (so DCIR doesn't go nuts due to transients).
- This task only starts after the first round of SOC's have been estimated for all modules (TaskUpdateSOC) gives a notification

void TaskADS1115(void *pvParameters)

- Interfacing with all peripherals like **ADS1115 (I2C)**, **NRF24 (SPI)**, **CAN**, etc are handled using state machines.
- Of course the Adafruit library exists for interfacing with the ADS1115, but it suffers from a flaw. It uses polling (the OS bit of the config register is polled continuously to check if the conversion is complete). However this isn't ideal for RTOS as other tasks could be run during the ADC conversion time. Hence an interrupt triggered, thread safe driver was built from scratch.
- Shared bus/resources are protected using **mutexes** (see *Resource management* section in the intro).
- ADS1115 conversions run in **single-shot mode**.
- The ESP32 starts conversions, and the I2C read is triggered by the **active-low RDY interrupt** on GPIO14.
- ADS1115 I2C address is 0b1001000 (0x48) since **ADDR is tied to GND**.

State 0 — Write config (setup)

- Write 0x0380 to **Config** register (pointer = 0x01).
- Bitfields:
 - OS (bit 15) = 0 (not starting conversion yet)
 - MUX (14:12) = 000 (AIN0 – AIN1)
 - PGA (11:9) = 001 (± 4.096 V, since HOB300 outputs ± 4 V)
 - MODE (8) = 1 (single-shot)
 - DR (7:5) = 100 (128 SPS default, not critical in single-shot)
 - COMP_MODE (4) = 0 (don't care for RDY)
 - COMP_POL (3) = 0 (active-low RDY)
 - COMP_LAT (2) = 0 (don't care for RDY)
 - COMP_QUE (1:0) = 00 (anything except 11 works)
- If the I2C write succeeds (all bytes ACK'd), go to **State 1**.

State 1 — Write Lo_thresh (enable RDY)

- Write 0x0000 to **Lo_thresh** (pointer = 0x02).
- RDY mode needs **MSB of Lo_thresh = 0**.
- If the I2C write succeeds, go to **State 2**.

State 2 — Write Hi_thresh (enable RDY)

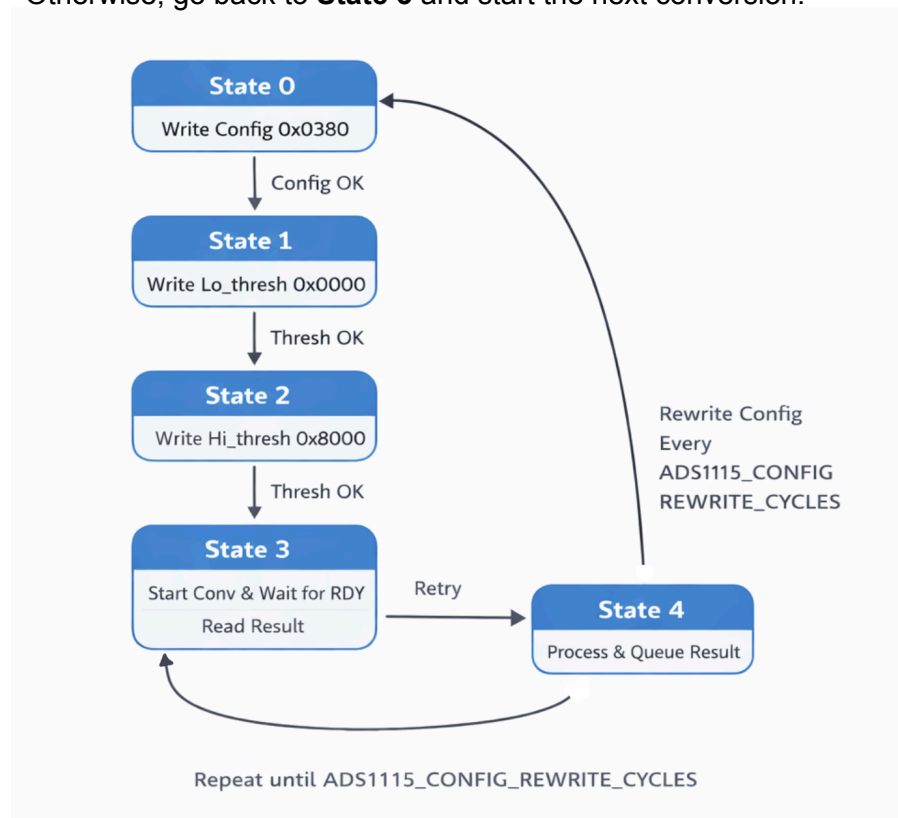
- Write 0x8000 to **Hi_thresh** (pointer = 0x03).
- RDY mode needs **MSB of Hi_thresh = 1**.
- If the I2C write succeeds, go to **State 3**.

State 3 — Start conversion + wait for RDY + read

- Write 0x8380 to **Config** (same as State 0, but OS = 1 to start conversion).
- Wait for the interrupt notification from the RDY pin.
 - **The same state that triggers conversion also waits for the notification.**
 - **This is intentional because the state machine has a 2 ms delay between states, and splitting the wait into another state could add up to ~2 ms of extra latency if the RDY interrupt arrives when the task isn't scheduled.**
- Once the notification is received from the ISR, write 0x00 to the pointer register to select **Conversion** register (pointer = 0x00).
- Request a 2-byte I2C read and read the conversion register.
- If 2 bytes are received successfully, go to **State 4**.

State 4 — Process + queue

- Read the 2 bytes from the I2C buffer into memory.
- Use the first 10 readings to estimate an average **zero-current bias**.
- Push the raw conversion value into the ADS1115Current queue.
- Every ADS1115_CONFIG_REWRITE_CYCLES (100 conversions), rewrite config from **State 0–2**.
 - Otherwise, go back to **State 3** and start the next conversion.



void TaskVAT4300(void *pvParameters)

- The VAT4300 shunt uses an **nRF24** link with a proprietary protocol.
- The protocol + config was reverse engineered by [Robert Flatley \(Youtube video\)](#)
- This task is basically a clean, **state-machine based**, **thread-safe** version of that approach.
- Implementation uses the **RF24 Arduino library**.

Comms scheme (state machine)

State 0 — Radio setup (address + frequency)

- The VAT4300 needs two critical settings:
 - VAT4300_address
 - VAT4300_frequency
- These are normally set by the original base station, but since that was lost, they were recovered by probing the **VAT4300 nRF24 SPI lines** on a scope + serial decode.
- The TX/RX pipes are opened (pipe 0) using the correct address and frequency.
- **Auto-ACK** is enabled for pipe 0.

State 1 — Transmit frame + wait for a valid RX (IRQ handling)

- Default TX payload format is:
 - 0xAA 0x04 0x01 <VAT4300_address> <28 bytes of 0x00>
- For the **first initialization frame**:
 - TX_payload[2] = 0xFF
 - TX_payload[4] = VAT4300_address + 0xAD
- For all **normal frames after init**:
 - TX_payload[2] = 0x01
 - TX_payload[4] = VAT4300_address + 0xAF
- The frame is transmitted, and then the radio switches to **listening mode**.

Important: NRF24 IRQ behavior

- IRQ (active-low) asserts for *multiple reasons*:
 - TX success (ACK received) (radio.write() returns true)
 - TX fail (MAX_RT) (radio.write() eventually returns false)
 - RX data received (RX_DR)
- In this task, IRQ is used only as a signal for **successful RX reception** from the VAT4300.
- Since TX events also trigger IRQ, the task clears any stale notification right after TX using:
 - xTaskNotifyStateClear(VAT4300_Task_Handle)

- After clearing, the task switches to RX mode and waits (max **1s**) for a valid RX notification.
- If no RX notification arrives:
 - Retransmit up to VAT4300_RESET_CONFIG_RETRIES
 - If still nothing, reset the radio config by going back to **State 0**

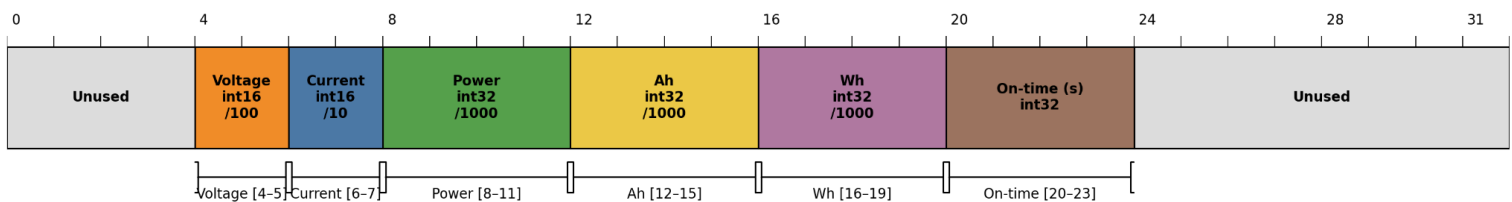
State 2 — Read RX payload

- Read the full **32-byte RX frame**.
- The first RX after TX init is ignored (has VAT4300 config data)

State 3 — Parse + push to queues

- Parse the RX frame and push decoded values to queues.
- Values received:
 - TS_voltage
 - VAT4300_current
 - TS_power
 - VAT4300 “on-time” (runtime / uptime)

VAT4300 nRF24 RX Payload (32 Bytes) — Byte Field Map



Note: Multi-byte fields are big-endian (MSB first).

void TaskBalancing(void *pvParameters)

- For each segment, the task finds the module with the **minimum SOC**.
- If any module has:
 - $SOC > (min_soc_of_segment + BALANCE_THRESHOLD)$
then that module’s balancing bit is set to 1 for that segment.
- The task overwrites the BAL queue (used by TaskLinduinoCVComms) with the latest balance config.

void TaskUpdateTSCurrent(void *pvParameters)

- The VAT4300 shunt current is trusted more than the fast-sampled Hall sensor (HBO300).
- If a new VAT4300 current value was recently updated by TaskVAT4300, TS_current is set to that value.
- Otherwise, the HBO300 value is used.

ADS1115 → HBO300 current conversion

- Raw ADS1115 readings are converted to current using:
 - HBO300_ADS1115_CURRENT_MULTIPLY_FACTOR
- The theoretical factor is:
 - ADS1115_resolution = 125 μ V/LSB (± 4.096 V range)
 - HBO300 scale = 300A / 4V
 - So:
 - $125e-6 * (300/4) = 0.009375$ A/LSB
- With real-world calibration this factor is set to:
 - 0.01 A/LSB

void TaskSDCFault(void *pvParameters)

- **This is one of the most important tasks.**
- It is the only user-defined task added to the **ESP WDT** (apart from the default WiFi background task).

Why this task exists (main failure mode)

- Tasks share data through queues using:
 - Queue overwrite for writes
 - Queue peek for non-destructive reads
- If a peripheral fails, the queue stops updating, **but still holds the last valid value**, which can silently hide failures.
- This is especially critical for the **isolated UART link** between ESP32 and Linduino (TaskLinduinoCVComms).

Fault checks (and which counter increments)

1) Linduino comms “stuck data” detection →

fault_counter[2]

- For each segment, the received cell codes are XOR'd together to form a per-segment “XOR signature”.
- The XOR signatures are compared against the previous cycle.
- If the XOR signature **does not change** while TS_current != 0, it indicates the cell data is no longer updating → UART link is likely down.
- In that case:
 - fault_counter[2]++

2) SOC out-of-bounds (cell V protection and SOC estimation check) →

fault_counter[0]

- If any cell/module SOC goes outside:
 - SOC_MIN_FAULT and SOC_MAX_FAULT
- Then:
 - fault_counter[0]++

3) Current sensor disagreement →

fault_counter[1]

- If:
 - $\text{abs}(I_{\text{VAT4300}} - I_{\text{HBO300}}) > \text{CURRENT_DEV_FAULT}$
- Then:
 - fault_counter[1]++

WDT behavior + fault latching

- As long as **all** fault counters are < 10, this task continues to reset the WDT.
- If **any** fault counter reaches >= 10:
 - All primary tasks are suspended
 - The SDC relay is opened
 - The task enters an infinite loop (so the WDT is no longer reset)
- Once the WDT expires, the ESP32 triggers a software reset.
- In setup(), the reset reason is checked:
 - If the reset was due to WDT timeout, setup() enters an infinite loop and the BMS stays disabled until a manual hardware reset.

This ensures fault latching: the SDC only closes again after faults are fixed and the BMS is manually reset.

void TaskTelnetStreamPrintDebug(void *pvParameters)

- This task provides a live **Telnet debug stream** for quick runtime inspection.
- The ESP32 connects to the local WiFi network and serves a Telnet stream (use **PuTTY** to connect).
- A simple menu is printed, and the user can type commands to continuously print values.

Menu

Instructions to print debug info

- Enter 1x to print SOC of x (0...) segment
- Enter 2x to print DCIR of x (0...) segment
- Enter 3x to print CellV of x (0...) segment
- Enter 4 to print VAT4300 current value
- Enter 5 to print ADS1115 raw values
- Enter 6 to print TS current value
- Enter 7 for current bias
- Enter 8 for VAT4300 on time
- Enter 0 for instructions

Example

- Entering for ex, "13", continuously prints SOC's of all modules in **segment 3 (0-5)**.
- 0 can be entered anytime to return to the main menu.

void TaskOTAHandleAlive(void *pvParameters)

- This task was originally used to connect to WiFi.
- If connection fails, it retries every OTA_RETRY_DELAY_MS.
- It keeps the OTA handle alive.
- Once WiFi is connected, it sends a start notification to TaskTelnetStreamPrintDebug.

void TaskCANComms(void *pvParameters)

- This task uses the **ESP32 TWAI CAN library** to transmit CAN messages.
- All data is **little-endian**.

CAN messages

1. ID = 0x5AA
 - 8 bytes total
 - float TS_current (4 bytes) + float TS_voltage (4 bytes)
2. ID = 0x5AB
 - 4 bytes total
 - float TS_power
3. ID = 0x5AC
 - 8 bytes total
 - float HBO300_current (4 bytes) + float VAT4300_current (4 bytes)
4. ID = 0x5AD
 - 7 bytes total
 - uint16_t DCIR_total_mOhm (2 bytes, sum of all modules)
 - float TS_voltage_total (4 bytes, pack voltage)
 - uint8_t SOC_avg (1 byte, avg of all modules)